

Skill 改写报告：从「裸 API 契约」到「中训语料」

对象技能：getHealthWorkforce（来源 [github:fairyshine/Seal-Tools](#)，类型 api_tool） |
改写流水线：SSM（Skill Spec Midtraining） | 语言：简体中文

一、这份报告说什么

本报告展示同一条 skill 在「改写器（rewriter）」前后的形态变化：左端是从开源工具集里抓到的原始素材——一段只描述参数与返回字段的裸 API 契约；右端是改写器最终产出的、可直接喂给基座模型做中训（midtraining）的自然语言文档。中间还有一层关键产物：把薄契约扩写成的五要素使用规范（Skill Spec）。三者的关系是一条改写链：

原始裸契约 → 五要素使用规范 → 多体裁中训语料

改写器不是简单翻译或扩字数，而是把「机器能解析的字段定义」改写成「智能体能内化的用法判断力」：什么时候该用、什么时候坚决别用、同类技能怎么选、会怎么用错、结果怎么核。

二、改写前：原始 skill 长什么样

智能体最初「看到」的就是下面这段 JSON。它信息很薄：只有参数类型和返回字段类型，没有任何使用判断——不会告诉你省略 `occupation` 会怎样、地名有歧义要先澄清、`doctor` 和 `physician` 是两个词、薪资数没标币种不能乱猜、大城市返回 0 几乎一定是参数错。

```
{
  "api_name": "getHealthWorkforce",
  "api_description": "Retrieve information about the health workforce in a specific location",
  "field": "Public Health/Health Workforce",
  "parameters": {
    "location": { "type": "str", "description": "想查询健康人力的地点（如国家、城市）" },
    "occupation": { "type": "str", "description": "健康工种（如 doctors、nurses、pharmacists）" },
  },
  "required": ["location"],
  "responses": {
    "health_workers": { "type": "int", "description": "该地点该工种的从业人数" },
  }
}
```

```
"occupation_salary": { "type": "float", "description": "该工种在该地点的平均薪资" },
"required_education":{ "type": "str",    "description": "该工种所需学历" }
}
}
```

问题：这种契约直接交给智能体，它能调用，但容易「死按字段、不会判断」——遇到歧义地点照样硬调、把缺参数的零值当事实上报。原始素材缺的正是「用法智慧」。

三、改写第一步：扩写成五要素使用规范（Skill Spec）

改写器先用一个模型把薄契约扩写成结构化的「使用规范」，拆成五个要素，每个要素再配若干条原子原则句（可独立判定真伪的最小判据）。这是整套方法的核心创新——把隐性的用法知识显式化。

要素	讲什么	关键原则（摘）
能力边界	它能做什么、边界在哪	只返回三个字段；不能改数据；不能跨域（能源治理 / 生物地理 / 供水）；省略 occupation 时返回字段语义未定义，不得臆断聚合含义。
何时别用	哪些请求不该调它	改 / 规划 / 更新数据 → 别用；跨域 → 别用；地点有歧义未澄清、或没指定职业导致输出不可解释 → 别调；静态常识且用户没要实时数据 → 自己答；历史趋势 / 预测 → 别用（只给当前快照）。
同类怎么选	和相似技能如何区分	读健康人力指标 → 本技能；改数据 → updateWorkforcePlan（写）；跨域 → 对应域技能。判据 = 领域 × 动作（读/写） × 是否需要新鲜权威数据。
失败模式	典型怎么用错、根因是什么	伦敦歧义（英国 vs 加拿大）；doctor 与 API 期望的 physician 术语错配 → 返回零；省略 occupation 得 0 误报「无从业者」；把薪资当某币种臆断；漏看某字段缺失。
结果	调用后怎么核对	校验三字段类型与合理域值；大城市出现 0 疑为参数错；与常识比对；refine 参数重试一次；仍异常就停手如实上报、建议官方核实，不无限重试、不乱切技能。

验证		
----	--	--

四、改写第二步：把规范改写成多体裁中训语料

改写器再把上面的规范，按不同体裁（审计报告、案例笔记、最佳实践备忘……）改写成自然语言文档。每篇定向其中一两个要素讲透，原则被自然复述成行文，而非编号罗列。下面这篇「使用行为审计报告」就是最终产出、要喂给基座模型做中训的语料（完整呈现）：

getHealthWorkforce 使用情况审计报告

审计重点。本次审计聚焦于近期代理调用 getHealthWorkforce 的行为，尤其关注一种反复出现的失败模式：代理在未提供 occupation 参数的情况下发起调用，随后将返回的零计数或不确定数据当作有效信息呈现给用户。所有发现均基于对调用轨迹的定性分析，不涉及统计抽样或量化比率。

发现：省略 occupation 导致的系统性误报。该技能的输出契约明确限定，health_workers、occupation_salary 和 required_education 三个字段的语义仅在提供 occupation 时有效。当 occupation 缺失，返回数据缺乏解释基准——零可能意味着「没有该职业的数据」而非「没有卫生工作者」。最常见的违规是代理忽略这一契约，在用户未指明职业时直接调用，继而把 health_workers 为零的结果报告为「该地没有卫生工作者」。这种误报很可能源于调用方假定缺失参数会触发某种汇总逻辑，但技能本身并不承诺此类行为，将未定义输出当作明确信息严重损害了回答的可靠性。

发现：位置歧义与术语不匹配加剧错误。多起调用中，代理未核实位置字符串的唯一性，将「伦敦」直接传入而未确认是英国还是加拿大，导致字段的地理归属存疑。职业命名与 API 预期术语不一致也屡见不鲜：用户说「医生」，代理就以「doctor」发送，而接口实际期待「physician」，结果返回零或异常值。只要在调用前补全这两个参数，异常零计数便大幅减少，说明失败主因并非数据源缺陷，而是调用方参数构造的疏忽。

发现：输出校验不完整与单位假设。即便参数完整，代理对返回数据的校验也常不足：有的响应仅返回部分字段，例如 required_education 为空字符串，代理却未标记缺失便径直交付；也有把 occupation_salary 直接当作某种货币解读，而未从响应中寻找单位线索，从而可能误导薪资比较。每一项数值输出都应被视作原始值，未经显式元数据确认前不得附加任何单位或货币推断。

建议。最紧迫的措施是在调用路径中强制 `occupation` 的存在：对任何未明确包含职业的请求，代理必须先追问具体岗位再发起调用，绝不在参数缺失时自行猜测。其次建立位置消歧的快速确认机制，对多义地名主动询问所属国家或地区；职业术语方面维护一套常用别名与 API 标准名的匹配表，匹配失败时向用户确认。每次调用返回后执行最低限度校验——`health_workers` 为非负整数、`occupation_salary` 为正数、`required_education` 为非空字符串；对大型知名城市出现的零计数视为红旗，澄清后重试一次，若仍不可信即终止调用、如实报告并建议查阅权威来源，而非陷入无意义的重试循环或转用无关技能。

说明：以上是「审计报告」这一正式体裁。改写器对同一条 `skill` 还会产出案例笔记、论坛问答等体裁——非正式体裁完全不提任何「规范／指南」，把原则化作团队经验与口口相传的规矩，以增加语料多样性。即便在可引用规范的正式体裁里，改写器也只允许至多一两次、且以作者自己的分析口吻陈述，避免「依照指南……」式的机械复读；审计类文字一律用定性量级（最常见、少量、偶尔）描述，绝不虚构样本量或百分比。

五、改写前后对比

维度	改写前（原始 skill）	改写后（中训语料）
形态	机器可解析的 JSON 字段定义	人类可读的自然语言文档（多体裁）
信息内容	只有参数 / 返回字段的类型与一句话描述	能力边界、禁用场景、同类选择、失败模式、结果验证及其理由
回答的问题	「这个接口接收什么、返回什么」	「该不该用、何时别用、怎么选、会怎么错、怎么核」
面向用途	让智能体能调用	让智能体先内化用法判断力，再去调用
典型风险	死按字段、遇歧义硬调、把零值当事实	—（这些风险正是文档要教模型规避的）

六、改写遵循的硬约束

为保证改写出的语料能真正提升模型判断力、而不是引入噪声，改写器受三条硬约束：

- 接地不编造。不虚构可验证的具体事实（人名、具体日期、引用、把没测过的数字当实测结果）。报告里的「某市」「零值」都是示意性举例，不是声称真实发生过的事件。
- 无术语回声。文档里不出现任何内部编号或「规范条目」式措辞；原则被自然复述成审计发现、整改建议或团队经验，让模型学到的是推理而非术语。
- 无微调轨迹格式。中训语料严禁出现 `<tool_call>`、`"arguments":`、`User:/Assistant:` 这类指令微调轨迹格式——中训要的是「散文里的推理」，把「教格式」留给后续的 SFT 阶段。

七、一句话总结

改写器把一条只说明字段的裸 API 契约，改写成了一份讲清如何明智使用它的自然语言文档：先经五要素规范把隐性用法知识显式化，再按多种体裁改写成中训语料。其目的是让基座模型在大量调用示范之前，先把每个 skill 的「用法判断力」内化成直觉。